ESCUELA DE FORMACIÓN DE TECNÓLOGOS

PROGRAMACION ORIENTADA A OBJETOS

ASIGNATURA:	POO
PROFESOR:	Ing. Yadira Franco R
PERÍODO ACADÉMICO:	2026A

TAREA S2

TÍTULO:

CLASES, OBJETOS, ATRIBUTOS Y METODOS INGRESO DE DATOS

Henry Paul Tonato Cajamarca

2026A

TEMA: CONSTRUCTORES

Actividad 1: Investigación

De **lo investigado poner ejemplo que ilustre el contenido para mayor comprensión**.

1. Convención, en Java Clases, atributos, métodos y objetos.

   Convención: Son las reglas de estilo recomendadas por la comunidad y Oracle para mantener el código uniforme y legible.

   Básicamente es el modelo de estructura para que todos puedan entenderlo, comenzando por las clases hasta llegar a sus objetos.

   Clases: Es el modelo, plano o plantilla genérica a partir de la cual se crean los objetos.

   Puede ser una clase Coche que puede definir elementos en general.

   Atributos: Son las variables que definen el estado o características de un objeto.

   De la Clase Coche sus atributos son las características como color, marca, tipo, etc.

   Métodos: Son funciones definidas dentro de una clase que describen el comportamiento o las acciones que un objeto puede realizar.

   Se podrían decir que son de la Clase Coche las acciones que este tiene como arrancar, frenar, acelerar, etc.

   Objetos: Es la entidad que vive en la memoria y contiene valores reales para los atributos definidos en la clase.

   Los objetos son el elemento conformado por los atributos y métodos de la Clase Coche que lo instancia de los demás.

Actividad 2: PRACTICA

Práctica 1: Clase Libro

Objetivo: Crear una clase con atributos, constructor y método para mostrar información.

Descripción:

- Crear la clase Libro con atributos: titulo, autor, anioPublicacion.
- Crear un constructor.
- Crear un método mostrarInformacion() que imprima los datos del libro.
- Crear 2 objetos de tipo Libro y mostrar su información.

```java
public class Main {
    public static void main(String[] args) {
        Libro libro1 = new Libro();

        libro1.autor="Henry";
        libro1.titulo="Software";
        libro1.anioPublicacion=2026;

        libro1.mostrarInformacion();

        Libro libro2 = new Libro();

        libro2.autor="Paul";
        libro2.titulo="Bombero";
        libro2.anioPublicacion=2030;

        libro2.mostrarInformacion();

    }
}
```

```java
public class Libro {  4 usages  new *
    String titulo, autor;  3 usages
    int anioPublicacion;  3 usages

    void mostrarInformacion(){  2 usages  new *
        System.out.println("Titulo: "+titulo);
        System.out.println("Autor: "+autor);
        System.out.println("Año de publicacion: "+anioPublicacion);
        System.out.println("==================================\n");
    }

}
```

```
Titulo: Software
Autor: Henry
Año de publicacion: 2026
==================================

Titulo: Bombero
Autor: Paul
Año de publicacion: 2030
==================================
```

Actividad 3: Tipos de Constructores

Un constructor en Java es un método especial que se ejecuta automáticamente al crear un objeto. Sirve para inicializar los atributos de la clase. Existen tres formas comunes de definirlo:

- Constructor con parámetros
- Constructor sin parámetros con valores directos
- Constructor vacío

Tarea práctica

Objetivo: Comprender y aplicar los tipos de constructores en Java.

Indicaciones:

1. Explica con tus propias palabras qué es un constructor.
   Un constructor es un método especial que se invoca automáticamente al crear un objeto a base de una palabra clave.

2. Crea una clase llamada Producto con tres atributos: nombre, precio, cantidad.
3. Implementa los tres tipos de constructores: uno con parámetros, uno con valores fijos, y uno vacío.
4. Crea tres objetos usando cada tipo de constructor.
5. Muestra los datos de cada objeto en consola.

Main.java × Producto.java

```java
public class Main {
    public static void main(String[] args) {

        Producto p1 = new Producto( nombre: "Laptop",  precio: 1200.50,  cantidad: 5);
        Producto p2 = new Producto();
        Producto p3 = new Producto( nombre: "Mousepad");

        System.out.print("Objeto 1: ");
        p1.mostrarDatos();
        System.out.print("Objeto 2: ");
        p2.mostrarDatos();
        System.out.print("Objeto 3: ");
        p3.mostrarDatos();

    }
}
```

Main.java Producto.java ×

```java
public class Producto {  6 usages  new *
    String nombre;  4 usages
    double precio;  4 usages
    int cantidad;  4 usages

    public Producto(String nombre, double precio, int cantidad) {  1 usage  new *
        this.nombre = nombre;
        this.precio = precio;
        this.cantidad = cantidad;
    }

    public Producto() {  1 usage  new *
        this.nombre = "Sin nombre";
        this.precio = 0.0;
        this.cantidad = 0;
    }

    public Producto(String nombre) {  1 usage  new *
        this.nombre = nombre;
        this.precio = 10.0; // Precio fijo
        this.cantidad = 1;  // Cantidad fija
    }

    void mostrarDatos(){  3 usages  new *
        System.out.println("\nNombre: "+nombre);
        System.out.println("Precio: "+precio);
        System.out.println("Cantidad: "+cantidad);
        System.out.println("===============\n");
    }

}
```

```
Objeto 1:
Nombre: Laptop
Precio: 1200.5
Cantidad: 5
===============

Objeto 2:
Nombre: Sin nombre
Precio: 0.0
Cantidad: 0
===============

Objeto 3:
Nombre: Mousepad
Precio: 10.0
Cantidad: 1
===============
```

Actividad 4: Simulación básica de una cuenta bancaria

Objetivo:

Aplicar los conceptos básicos de Programación Orientada a Objetos (POO) creando una clase con atributos privados, constructor público y tres métodos propios públicos.

Enunciado:

Imagina que estás desarrollando un programa sencillo para un banco. Cada cliente puede abrir una cuenta, consultar su saldo, depositar dinero y retirar dinero.

Tu tarea es crear una clase llamada CuentaBancaria que permita reutilizar la información del cliente y realizar acciones mediante métodos propios.

Requisitos técnicos:

6. Crear la clase CuentaBancaria con los siguientes atributos privados:
   - nombreTitular (String)
   - saldo (double)
7. Crear un constructor público que permita inicializar los datos del titular y el saldo inicial.
8. Implementar los siguientes métodos propios públicos:
   - mostrarInformacion(): Muestra el nombre del titular y el saldo actual.
   - depositar(double monto): Aumenta el saldo.
   - retirar(double monto): Disminuye el saldo si hay suficiente dinero.
9. En la clase principal (Main):
   - Crear un objeto de tipo CuentaBancaria.
   - Llamar a los tres métodos en orden lógico.
   - Mostrar cómo se reutiliza la información del objeto en cada acción.

¿Qué acciones realiza el objeto?

El objeto creado a partir de la clase CuentaBancaria puede realizar acciones a través de sus métodos propios. Estas acciones son:

- mostrarInformacion(): muestra los datos almacenados en el objeto.
- depositar(): modifica el saldo sumando un monto.
- retirar(): modifica el saldo restando un monto si hay fondos suficientes.

Estas acciones se ejecutan desde la clase principal usando el objeto. Por ejemplo:

```
CuentaBancaria cuenta1 = new CuentaBancaria("Lucía", 100.0);
cuenta1.mostrarInformacion();
cuenta1.depositar(50.0);
cuenta1.retirar(30.0);
```

Main.java × CuentaBancaria.java

```java
public class Main {
    public static void main(String[] args) {

        CuentaBancaria cuenta = new CuentaBancaria( nombreTitular: "Juan Pérez",  saldoInicial: 1000.0);

        System.out.println("--- Estado Inicial ---");
        cuenta.mostrarInformacion();

        System.out.println("\n--- Realizando Operaciones ---");
        cuenta.depositar( monto: 500.0);
        cuenta.retirar( monto: 200.0);

        System.out.println("\n--- Estado Final ---");
        cuenta.mostrarInformacion();
    }
}
```

Main.java CuentaBancaria.java ×

```java
public class CuentaBancaria {  2 usages  new *
    private String nombreTitular;  2 usages
    private double saldo;  5 usages

    public CuentaBancaria(String nombreTitular, double saldoInicial) {  1 usage  new *
        this.nombreTitular = nombreTitular;
        this.saldo = saldoInicial;
    }

    public void mostrarInformacion() {  2 usages  new *
        System.out.println("Titular: " + nombreTitular + " | Saldo: $" + saldo);
    }

    public void depositar(double monto) {  1 usage  new *
        if (monto > 0) {
            saldo += monto;
            System.out.println("Depósito de $" + monto + " exitoso.");
        }
    }

    public void retirar(double monto) {  1 usage  new *
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            System.out.println("Retiro de $" + monto + " exitoso.");
        } else {
            System.out.println("Saldo insuficiente para retirar $" + monto);
        }
    }
}
```

```
--- Estado Inicial ---
Titular: Juan Pérez | Saldo: $1000.0

--- Realizando Operaciones ---
Depósito de $500.0 exitoso.
Retiro de $200.0 exitoso.

--- Estado Final ---
Titular: Juan Pérez | Saldo: $1300.0
```

TEMA: ENCAPSULAMIENTO

"Encapsular es proteger los datos y controlar quién los puede ver o cambiar."

1. Que son los modificadores de accesos y sus tipos con ejemplos

Modificador	Acceso permitido
`public`	Desde cualquier clase
`protected`	Desde el mismo paquete y subclases
(default)	Solo dentro del mismo paquete
`private`	Solo dentro de la misma clase

2. Diferencia entre método propio y método constructor con sus ejemplos.

- **Método Constructor:** Especial, nombrado igual que la clase, sin tipo de retorno void se invoca automáticamente con new para inicializar atributos.
- **Método Propio (o de instancia):** Define acciones o comportamientos específicos del objeto. Tiene nombre personalizado, retorno obligatorio (aunque sea void) y se llama manualmente.

```
public class Coche {
   String marca;
   // CONSTRUCTOR: Inicializa el objeto
   public Coche(String marca) {
      this.marca = marca;
   }
   // MÉTODO PROPIO: Comportamiento
   public void acelerar() {
      System.out.println("El " + marca + " acelera");
   }
}
```

Uso: Coche miCoche = new Coche("Toyota"); <- Constructor

miCoche.acelerar(); <- Método propio

3. ¿Cómo se crea un objeto? ¿Por qué se crean objetos?, ¿Cuándo debo crear un objeto?

Se crean utilizando la palabra clave new seguida del llamado al constructor de la clase: Clase objeto = new Clase();

Se crean para representar entidades del mundo real en el código, gestionar estado (datos) y comportamiento (acciones) de manera modular.

Se pueden crear cuando necesites una instancia única con sus propios datos. Por ejemplo, al leer datos de una base de datos o al requerir realizar acciones específicas con atributos particulares.

4. Que es encapsulamiento ventajas y aplicación.

El encapsulamiento es el principio de la POO que oculta el estado interno de un objeto, limitando el acceso directo a los atributos y exponiendo solo métodos autorizados (getters y setters).

- **Ventajas:**
  - **Protección de Datos:** Evita modificaciones accidentales de los atributos.
  - **Modularidad:** Permite cambiar la implementación interna sin afectar al código externo.
  - **Validación:** Se pueden incluir reglas en los métodos set (ej. if para validar una edad).
- **Aplicación:**
    1. Declarar atributos como private.
    2. Crear métodos públicos get (obtener) y set (establecer/modificar).

```
public class Persona {
   private int edad; // Oculto

   public int getEdad() { return edad; }
   public void setEdad(int nuevaEdad) {
      if (nuevaEdad > 0) this.edad = nuevaEdad; <- Validación
   }
}
```

MODIFICADORES DE ACCESO

Modificador	Ejemplo real	¿Quién accede?
`public`	Mochila abierta	Todos
`private`	Mochila con clave	Solo tú
`protected`	Familia cercana	Algunos
(default)	Casa	Solo los de adentro

No todo debe estar abierto → porque pueden dañarlo

- ¿Qué hace private?

Es un modificador de acceso que oculta atributos y métodos. Al declarar un miembro como private, se impide que el código externo a la clase acceda o modifique directamente el estado interno, asegurando que solo los métodos de la propia clase lo manipulen.

- ¿Qué es encapsular?

Es el principio de la Programación Orientada a Objetos (POO) que combina datos (atributos) y los métodos que operan sobre ellos en una clase, protegiéndolos de manipulaciones externas no autorizadas y limitando la complejidad.

- ¿Para qué sirve un get?

Es un método público (getter) utilizado para obtener o leer el valor de un atributo privado. Proporciona una interfaz segura para acceder a la información sin exponer directamente la variable.

- ¿Para qué sirve un set?

Es un método público (setter) utilizado para asignar o modificar el valor de un atributo privado. Permite controlar cómo se actualiza el dato.

- ¿Por qué validar?

Al usar métodos set, se pueden implementar reglas de negocio antes de actualizar un dato. Esto garantiza la integridad y consistencia: por ejemplo, no permitir una edad negativa, asegurar que un campo no esté vacío o validar formatos.

```
private String nombre;
private double precio;
private int cantidad;
```

```
public class CuentaBancaria {
    private String nombreTitular;
    private double saldo;  5 usages
```

ENTREGA

- Subir sus desarrollos de Java a GitHub, en el archivo README PONER el texto de los ejercicios que va a desarrollar.

  Link GitHub: https://github.com/AltHenry29/Tarea3.git

- Crear el informe en el documento Word, incluir capturas de pantalla como evidencia y agregar el enlace del repositorio en el PDF, subirlo al AULA VIRTUAL.